C1_Week1

Machine Learning - Học Máy

Các thể loại Machine Learning:

- Supervised Learning
- Unsupervised Learning
- Recommender Systems
- Reinforcement Learning

Supervised Learning

Học mà từ input X → Y để từ đó có thể dự đoán từ các input mới

Có 2 loại Supervised Learning:

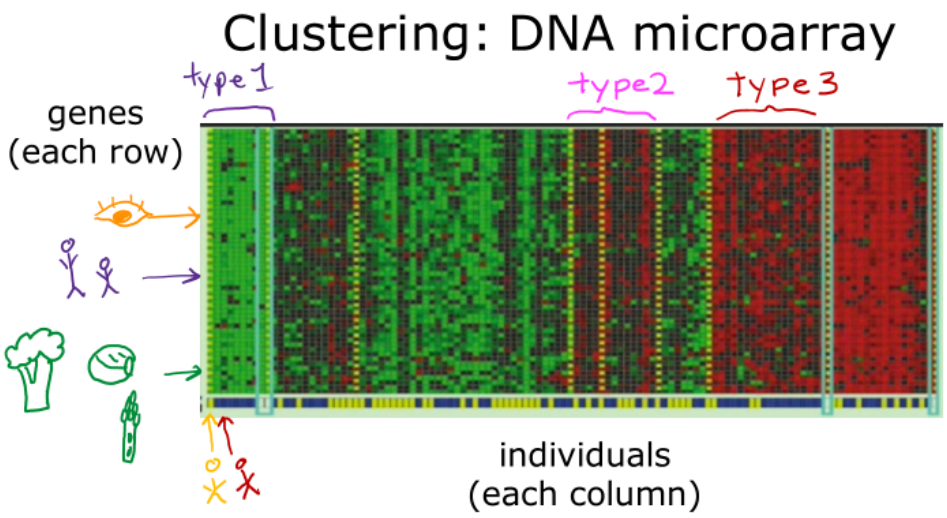
- Regression:
  - Dự đoán một số, một model regression không có giới hạn trong các con số mà nó dự đoán
  - Ví dụ: dự đoán giá của một căn nhà khi có kích thước.
- Classification:
  - Phân loại, hoặc đơn giản hơn là dự đoán vào các nhóm output hữu hạn
  - Ví dụ: dự đoán(phân loại) xem khối u lành tính hay ác tính trong bệnh nhân ung thư vú.

Unsupervised Learning

Giống supervised nhưng không có Y, máy phải tự tìm các pattern từ các thông tin input cho trước, giống như phân chia nhóm (Clustering), phát hiện bất thường(Anomaly). Hiểu đơn giản là tìm một cái gì đó “thú vị” từ data ấy.

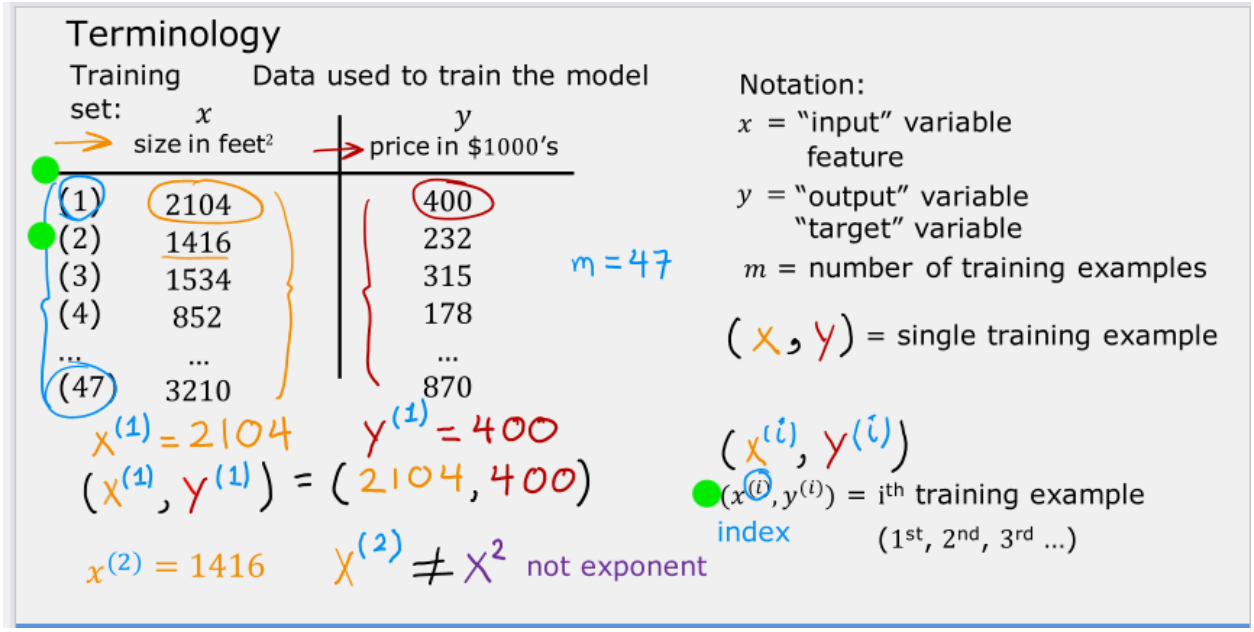
Ví dụ:

- Phân nhóm ở trong Google News từ các đề mục
- Phân loại khách hàng
- Phân nhóm trong DNA microarray



Linear Regression

- Linear là "tuyến tính", ở đây là việc mình vẽ một phương trình đường thẳng (hoặc là việc có thể được biểu diễn thành đường thẳng?)
- Regression là "hồi quy", việc dự đoán một giá trị (không rõ lắm?)
- Một số ký hiệu quan trọng:



→ Về bản chất của một mô hình Linear Regression, mình sẽ cần một training set để huấn luyện cho một learning algorithm. Thuật toán ấy (phương trình tuyến tính) từ đó có thể được sử dụng, chuyển một input (feature) thành một prediction

$$x \rightarrow f_{w,b} \rightarrow \hat{y}$$

feature → model → prediction

Trong đấy:

- x là training set
- f_w,b phương trình tuyến tính f(x) = wx + b
- ŷ là prediction từ model của mình

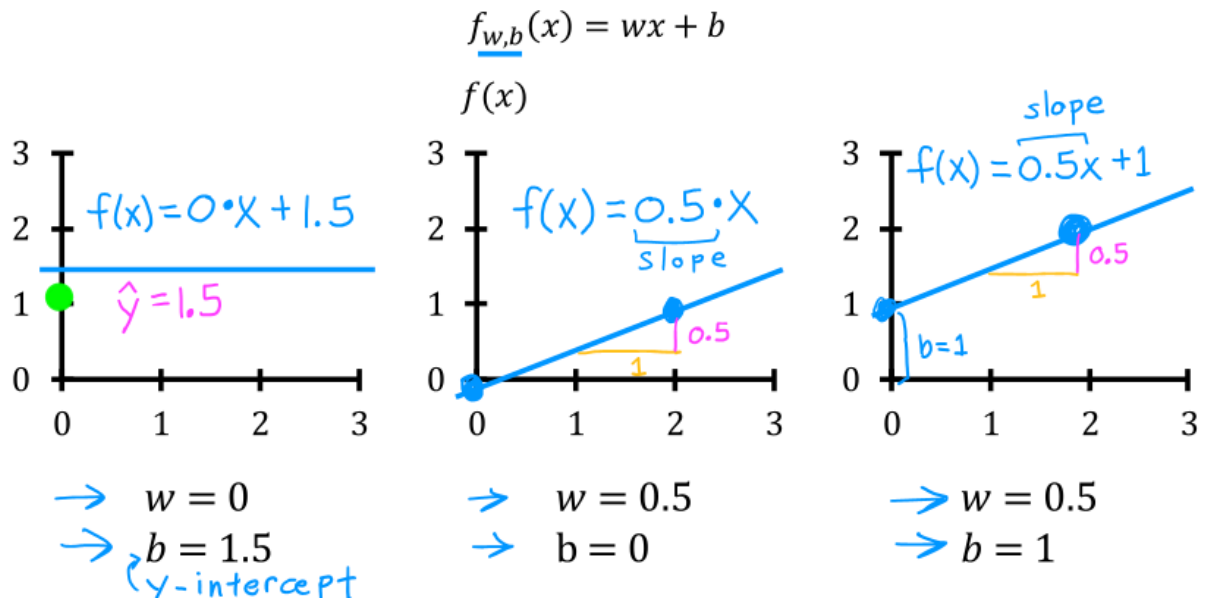
Phân tích rõ hơn về model của mình:

$$f_{w,b} = wx + b$$

- w (weight) là trọng số, na ná với hệ số góc trong phương trình y = ax + b

- b (bias) là độ lệch, b điều khiển độ cao thấp khi vẽ đồ thị
- Cả w và b được gọi là tham số (parameters)

→ Việc mình train một model chính là việc mình tìm ra bộ parameters phù hợp sao cho hợp với dữ liệu cho trước nhất có thể.



Một số ảnh hưởng tới đồ thị của mô hình, phụ thuộc vào các giá trị parameters khác nhau.

Đến đây mình cần đánh giá xem liệu model của mình có hiệu quả không, nên mình sẽ tạo ra một Cost Function

---

Cost Function

Nhắc lại Model của mình:

$f_{w,b}(x^{(i)}) = wx^{(i)} + b$

Mình có thể việt lại thành prediction y hat:

$\hat{y}^{(i)} = f_{w,b}(x^{(i)})$

Cost Function (Squared error cost function)

$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$

- Khi viết đầy đủ ra:

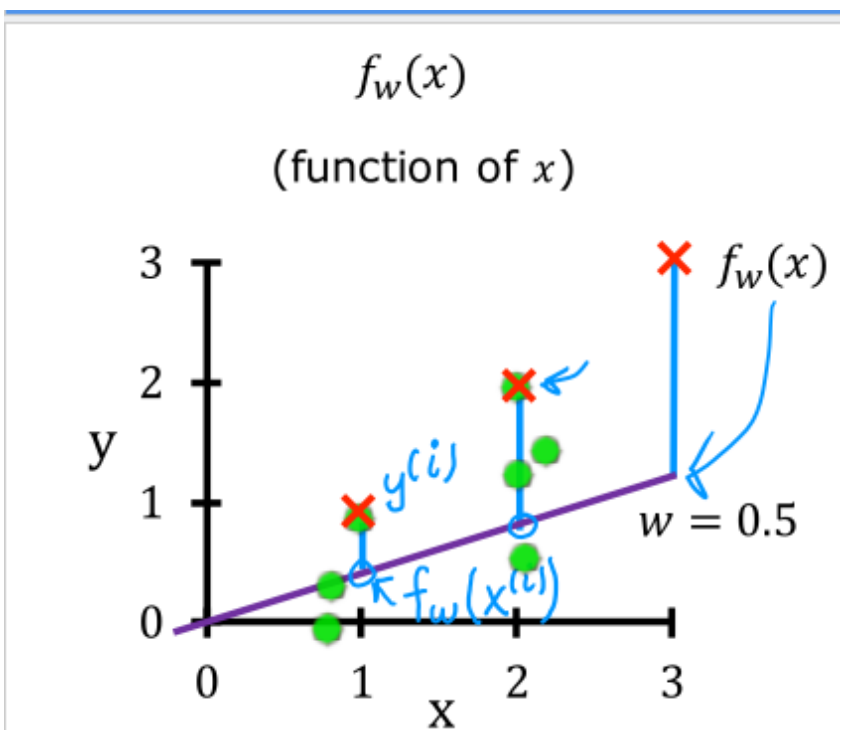
$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2$

- Hàm cost function sẽ tính tổng tất cả các cost của từng training example trong training set, bình phương lên và chia cho 2 lần tổng số training examples (2m)
    - Lý do mình chia 2 là do sau này khi mình đạo hàm để sử dụng gradient descent, số 2 sẽ được triệt tiêu
    - Việc bình phương cũng giúp cho loại bỏ các cost nhỏ hơn không, mặt khác khiến các cost càng lớn sẽ lớn hơn nữa (vì bình phương)

Khi mình trừ đi:

$$f_{w,b}(x^{(i)}) - y^{(i)}$$

đồng nghĩa với việc mình đang tìm khoảng các từ target tới dự đoán của mình vậy



→ Như đã viết bên trên, mình cần tìm cặp w, b sao cho prediction y hat gần với target nhất có thể.

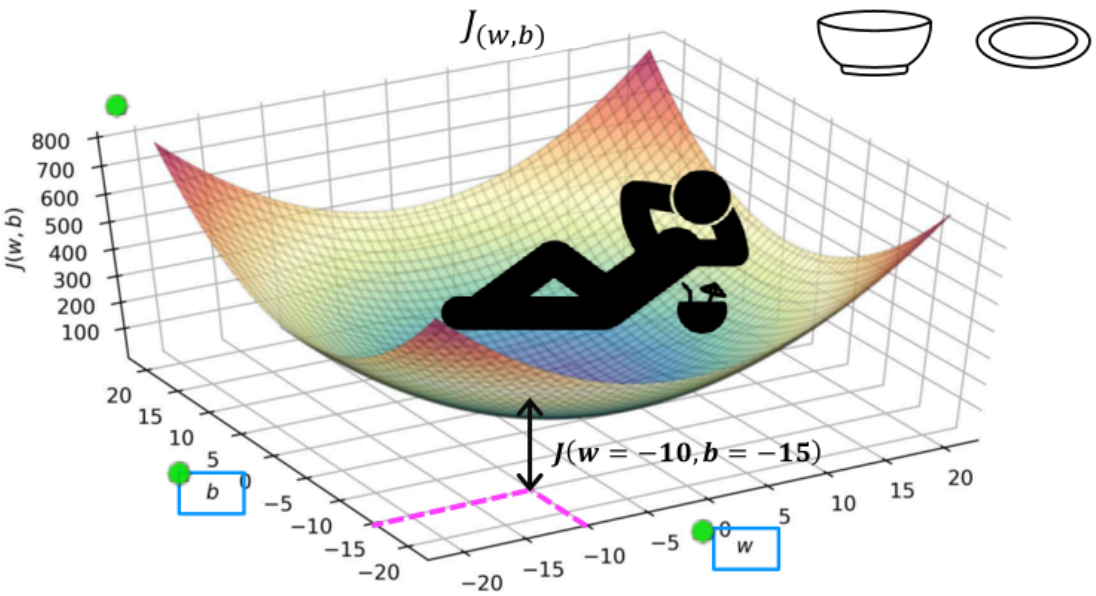
$$\text{Find } w, b : \hat{y}^{(i)} \text{ is close to } y^{(i)} \text{ for all } (x^{(i)}, y^{(i)})$$

Visualizing the Cost Function

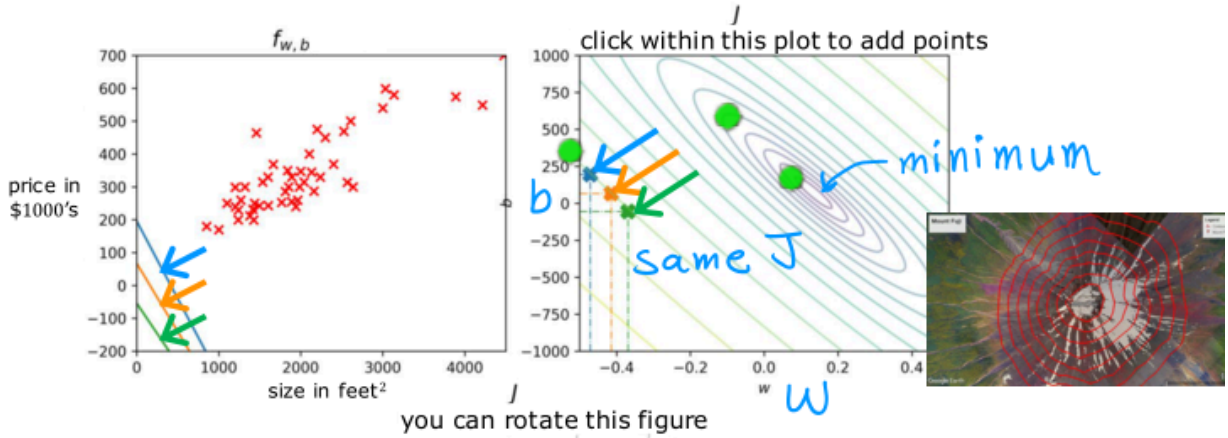
Nhắc lại Cost Function:

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

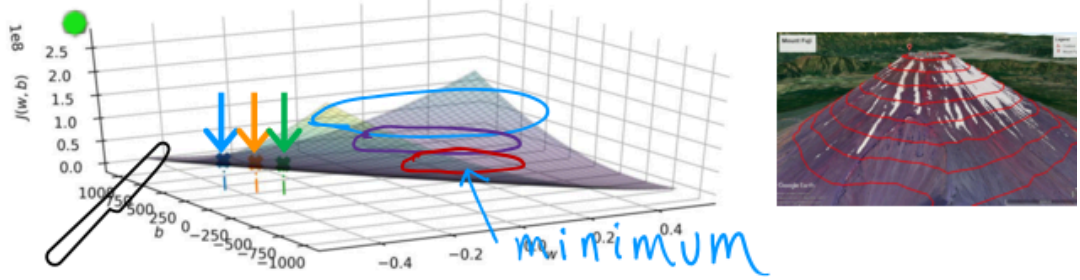
- Giả sử giá trị của w và b nằm trong [-20, 20], trục đứng của đồ thị làm J(w,b), ta có hình vẽ:

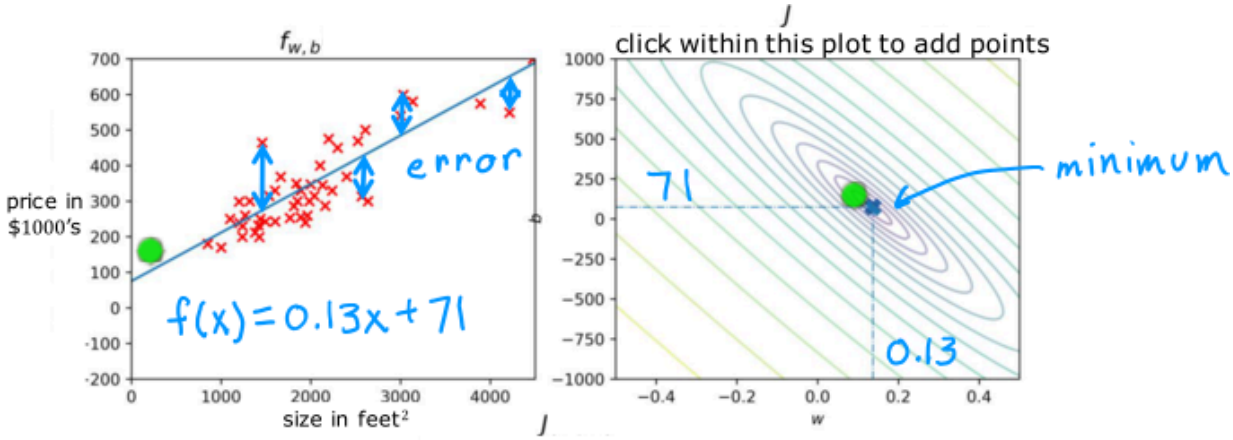


- Graph của mình có dạng convex lồi, và nhận thấy nó có một cực tiểu duy nhất (Global Minimum), nên mình không sợ rằng quá trình học máy vấp phải những Local Minimum.
- Để sau này mình có thể hình dung rõ hơn đồ thị này, mình sẽ dùng các Contours (đường đồng mức) để giúp hiển thị rõ hơn giá trị, với mỗi hình elip có "cùng độ cao"(chung giá trị)(hoặc là cost bằng nhau)

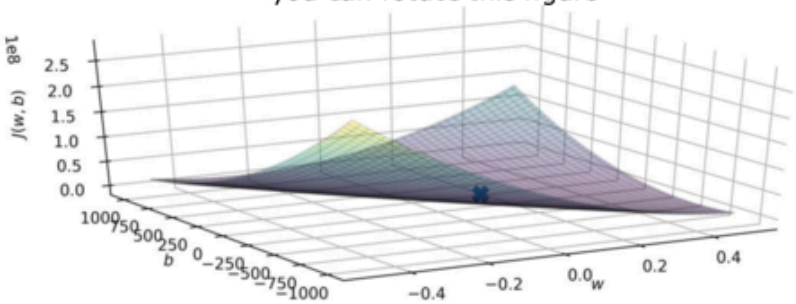


you can rotate this figure



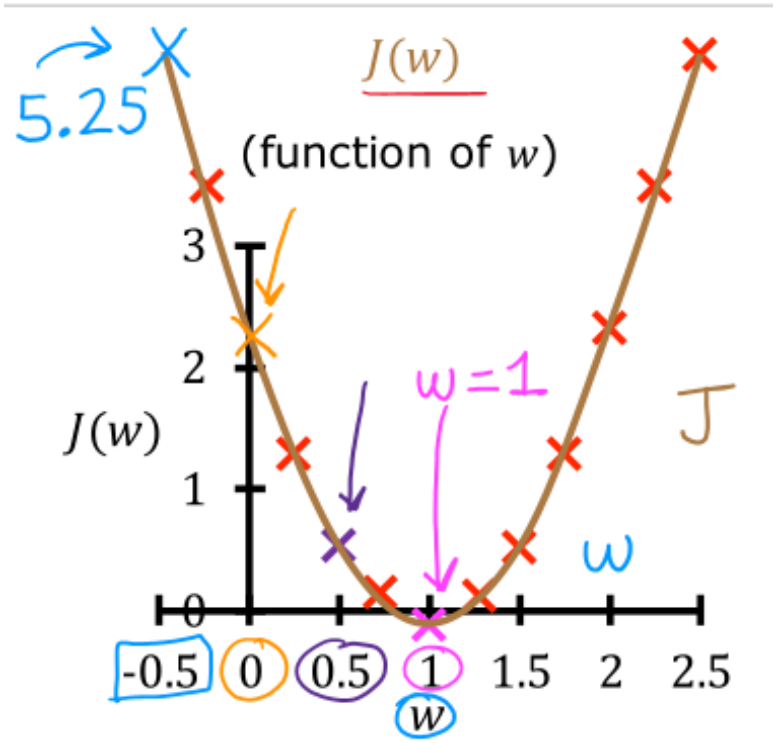


you can rotate this figure



- Note:

- Nếu mình biết trước một cặp (w,b) thì mình sẽ chỉ biết MỘT ĐIỂM trên đồ thị.
- Nếu mình biết được một giá trị (ví dụ b = 0) , thì parameter còn lại với J(w,b) sẽ được vẽ thành một hình parabol



Giả sử với b = 0

Gradient Descent

- Gradent Descent chính là thuật toán giúp tối ưu hoá w, b, chạy lặp đi lặp lại sao cho Cost Function đạt giá trị nhỏ nhất.
- Gradient Descent có thể được sử dụng cho bất cứ Model nào, không chỉ giới hạn ở Linear Regression.
- Các bước thực hiện Gradient Descent:
  - Bắt đầu ở một giá trị w, b (có thể ở w = 0, b = 0)
  - Thay đổi w và b qua từng vọng lặp liên tục để giảm Cost Function J(w,b)
  - Thay đổi cho tới khi J(w,b) đạt được cực tiểu.

→ Để thay đổi được w và b , ta có các công thức:

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$w = tmp_w$$

$$b = tmp_b$$

- Trong đó:
  - Alpha(**α)** là learning rate (hoặc step size). α sẽ quyết định việc qua mỗi lần lặp Gradient Descent sẽ đi xuống (hoặc đi lên) bao xa. Nếu α nhỏ thì Gradient Descent sẽ chậm, nếu α quá lớn thì Gradient Descent sẽ có thể nhảy qua(Overshoot) cực tiểu, dẫn đến việc không thể converge
  - tmp_w và tmp_b là các biến tạm của w và b được tính bởi THAM SỐ CŨ
  - Nhớ rằng đạo hàm bản chất là để tính độ dốc. Phần đạo hàm đằng sau là để tính liệu xem phải chỉnh w và b như thế nào, bởi vì đạo hàm được nhân với Learning Step.
- Mình có thể viết cụ thể sau khi đã đạo hàm:

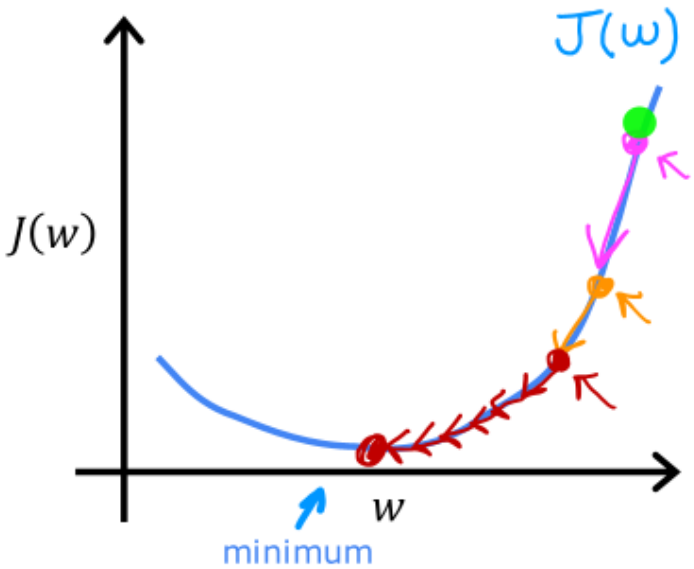
$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

Note:

- Để ứng dụng được đúng Gradient Descent trong code, mình phải CẬP NHẬT ĐỒNG THỜI (Simultaneous Update), nghĩa là mình phải tính biến tạm bằng THAM SỐ CŨ, xong cập nhật cả 2 w và b cùng lúc.

- Mặc dù khi chạy Gradient Descent, learning rate không đổi, nhưng đạo hàm thay đổi. Khi càng tới gần cực tiểu đồ thị, đạo hàm sẽ càng trở nên nhỏ hơn, nên dẫn tới các bước nhảy của cũng nhỏ hơn.



(Optional)

$$\frac{\partial}{\partial w} J(w,b) = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right)^2$$

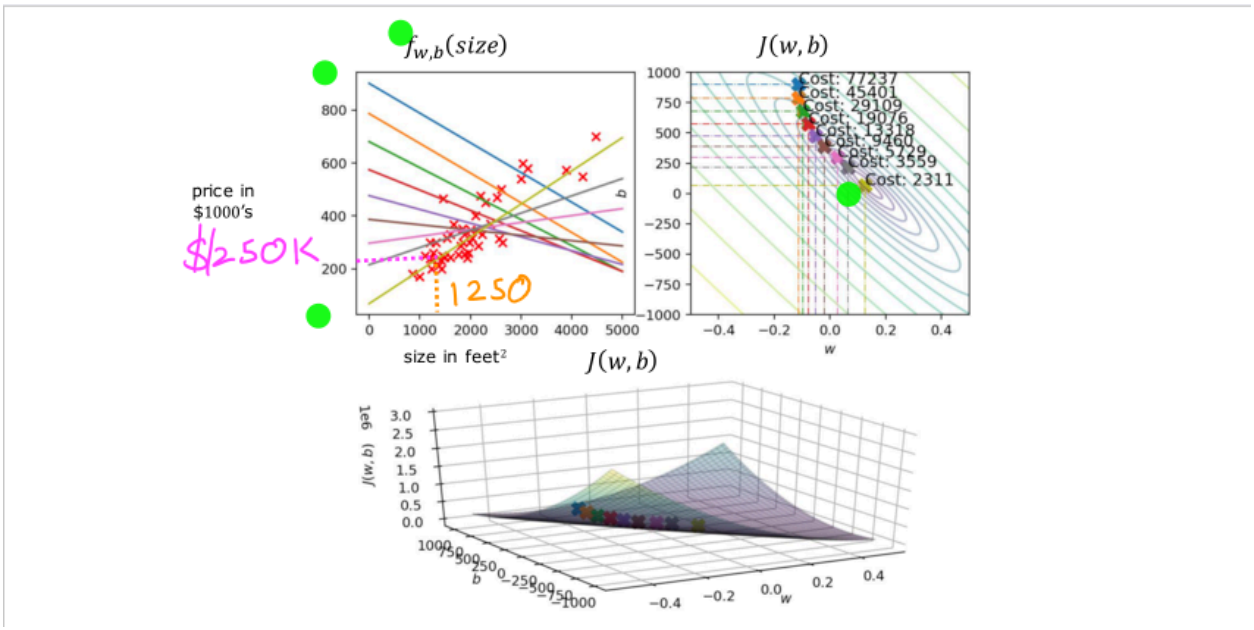
$$= \frac{1}{\cancel{2}m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right) \cancel{2} x^{(i)} = \boxed{\frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right) x^{(i)}}$$

$$\frac{\partial}{\partial b} J(w,b) = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right)^2$$

$$= \frac{1}{\cancel{2}m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right) \underbrace{\cancel{2}}_{\text{no } x^{(i)}} = \boxed{\frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)}$$

Slide chứng minh công thức đạo hàm

Khi mình chạy Gradient Descent các bước nhảy sẽ được minh hoạ như:



- Có thể thấy giữa các bước nhảy, các cost được giảm xuống dần dần và đến gần với cực tiểu, cũng như là phương trình đường thẳng của mình được chỉnh hoá sao cho phù hợp nhất với Training Set